

Adopt Library Fundamentals V1 TS Components for C++17

There are currently several proposals in flight to adopt portions of the Library Fundamentals Technical Specification for C++17.

The difficulty of such a piecemeal approach is twofold. It adds a lot of unnecessary overhead to process many proposals rather than a single proposal. It may cause useful but unexciting components to get dropped on the floor because no one notices or because no one is willing to go through the drudgery of writing yet another proposal and shepherding it through the committee.

This paper proposes that the components of TS 19568, *C++ Extensions for Library Fundamentals*, listed below be adopted into the C++17 working paper. The list was initialized with all of the components from [N4480, the Library Fundamentals DTS](#). The intent is that LWG/LEWG will strike any components they do not want included in C++17 or wish to process as separate proposals.

- 2 [Modifications to the C++ Standard Library](#)
 - 2.1 [Uses-allocator construction](#)
- 3 [General utilities library](#)
 - 3.1 [Utility components](#)
 - 3.1.1 [Header <experimental/utility> synopsis](#)
 - 3.1.2 [Class erased_type](#)
 - 3.2 [Tuples](#)
 - 3.2.1 [Header <experimental/tuple> synopsis](#)
 - 3.2.2 [Calling a function with a tuple of arguments](#)
 - 3.3 [Metaprogramming and type traits](#)
 - 3.3.1 [Header <experimental/type_traits> synopsis](#)
 - 3.3.2 [Other type transformations](#)
 - 3.4 [Compile-time rational arithmetic](#)
 - 3.4.1 [Header <experimental/ratio> synopsis](#)
 - 3.5 [Time utilities](#)
 - 3.5.1 [Header <experimental/chrono> synopsis](#)
 - 3.6 [System error support](#)
 - 3.6.1 [Header <experimental/system_error> synopsis](#)
- 4 [Function objects](#)
 - 4.1 [Header <experimental/functional> synopsis](#)
 - 4.2 [Class template function](#)
 - 4.2.1 [function construct/copy/destroy](#)
 - 4.2.2 [function modifiers](#)
 - 4.3 [Searchers](#)
 - 4.3.1 [Class template default_searcher](#)
 - 4.3.1.1 [default_searcher creation functions](#)
 - 4.3.2 [Class template boyer_moore_searcher](#)
 - 4.3.2.1 [boyer_moore_searcher creation functions](#)
 - 4.3.3 [Class template boyer_moore_horspool_searcher](#)
 - 4.3.3.1 [boyer_moore_horspool_searcher creation functions](#)
- 5 [Optional objects](#)
 - 5.1 [In general](#)
 - 5.2 [Header <experimental/optional> synopsis](#)
 - 5.3 [optional for object types](#)
 - 5.3.1 [Constructors](#)
 - 5.3.2 [Destructor](#)
 - 5.3.3 [Assignment](#)
 - 5.3.4 [Swap](#)
 - 5.3.5 [Observers](#)
 - 5.4 [In-place construction](#)
 - 5.5 [No-value state indicator](#)
 - 5.6 [Class bad_optional_access](#)
 - 5.7 [Relational operators](#)
 - 5.8 [Comparison with nullopt](#)
 - 5.9 [Comparison with T](#)
 - 5.10 [Specialized algorithms](#)
 - 5.11 [Hash support](#)
- 6 [Class any](#)
 - 6.1 [Header <experimental/any> synopsis](#)

- 6.2 [Class bad_any_cast](#)
- 6.3 [Class any](#)
 - 6.3.1 [any construct/destruct](#)
 - 6.3.2 [any assignments](#)
 - 6.3.3 [any modifiers](#)
 - 6.3.4 [any observers](#)
- 6.4 [Non-member functions](#)
- 7 [string view](#)
 - 7.1 [Header <experimental/string_view> synopsis](#)
 - 7.2 [Class template basic_string_view](#)
 - 7.3 [basic_string_view constructors and assignment operators](#)
 - 7.4 [basic_string_view iterator support](#)
 - 7.5 [basic_string_view capacity](#)
 - 7.6 [basic_string_view element access](#)
 - 7.7 [basic_string_view modifiers](#)
 - 7.8 [basic_string_view string operations](#)
 - 7.8.1 [Searching basic_string_view](#)
 - 7.9 [basic_string_view non-member comparison functions](#)
 - 7.10 [Inserters and extractors](#)
 - 7.11 [Hash support](#)
- 8 [Memory](#)
 - 8.1 [Header <experimental/memory> synopsis](#)
 - 8.2 [Shared-ownership pointers](#)
 - 8.2.1 [Class template shared_ptr](#)
 - 8.2.1.1 [shared_ptr constructors](#)
 - 8.2.1.2 [shared_ptr observers](#)
 - 8.2.1.3 [shared_ptr casts](#)
 - 8.2.2 [Class template weak_ptr](#)
 - 8.2.2.1 [weak_ptr constructors](#)
 - 8.3 [Type-erased allocator](#)
 - 8.4 [Header <experimental/memory_resource> synopsis](#)
 - 8.5 [Class memory_resource](#)
 - 8.5.1 [Class memory_resource overview](#)
 - 8.5.2 [memory_resource public member functions](#)
 - 8.5.3 [memory_resource protected virtual member functions](#)
 - 8.5.4 [memory_resource equality](#)
 - 8.6 [Class template polymorphic_allocator](#)
 - 8.6.1 [Class template polymorphic_allocator overview](#)
 - 8.6.2 [polymorphic_allocator constructors](#)
 - 8.6.3 [polymorphic_allocator member functions](#)
 - 8.6.4 [polymorphic_allocator equality](#)
 - 8.7 [template alias resource_adaptor](#)
 - 8.7.1 [resource_adaptor](#)
 - 8.7.2 [resource_adaptor_imp constructors](#)
 - 8.7.3 [resource_adaptor_imp member functions](#)
 - 8.8 [Access to program-wide memory_resource objects](#)
 - 8.9 [Pool resource classes](#)
 - 8.9.1 [Classes synchronized_pool_resource and unsynchronized_pool_resource](#)
 - 8.9.2 [pool_options data members](#)
 - 8.9.3 [pool_resource constructors and destructors](#)
 - 8.9.4 [pool_resource members](#)
 - 8.10 [Class monotonic_buffer_resource](#)
 - 8.10.1 [Class monotonic_buffer_resource overview](#)
 - 8.10.2 [monotonic_buffer_resource constructor and destructor](#)
 - 8.10.3 [monotonic_buffer_resource members](#)
 - 8.11 [Alias templates using polymorphic memory resources](#)
 - 8.11.1 [Header <experimental/string> synopsis](#)
 - 8.11.2 [Header <experimental/deque> synopsis](#)
 - 8.11.3 [Header <experimental/forward_list> synopsis](#)
 - 8.11.4 [Header <experimental/list> synopsis](#)
 - 8.11.5 [Header <experimental/vector> synopsis](#)
 - 8.11.6 [Header <experimental/map> synopsis](#)
 - 8.11.7 [Header <experimental/set> synopsis](#)
 - 8.11.8 [Header <experimental/unordered_map> synopsis](#)
 - 8.11.9 [Header <experimental/unordered_set> synopsis](#)
 - 8.11.10 [Header <experimental/regex> synopsis](#)
- 9 [Futures](#)
 - 9.1 [Header <experimental/future> synopsis](#)
 - 9.2 [Class template promise](#)
 - 9.3 [Class template packaged_task](#)
- 10 [Algorithms library](#)

- 10.1 [Header <experimental/algorithm> synopsis](#)
- 10.2 [Search](#)
- 10.3 [Shuffling and sampling](#)

Directions to the C++ working paper project editor

Add the components in the above list to the C++ *working paper* using the content for each component from the latest version of the Library Fundamentals V2 working paper. Move this content from the

`std::experimental::fundamentals_v2` namespace to the `std` namespace.

Note: Although we are adopting only the specified Library Fundamentals V1 components into the C++ working paper, the wording from those components comes from Library Fundamentals V2 to ensure the wording includes the latest corrections.
